SYDE 671 · ASSIGNMENT 2 · FALL 2026

Spatial Filtering and Multiresolution Blending

Moksh Trehan · Assignment 2 · Updated October 19, 2026

This report examines convolution, image gradients, unsharp masking, hybrid images, and multiresolution blending. Numerical comparisons and visual results are used to evaluate the implementations.

Experiments

NumPy convolution · Finite differences · Derivatives of Gaussians · HSV gradient orientations · Unsharp masking · Three hybrid images · Gaussian and Laplacian stacks · Oraple and two custom blends · Color comparisons

Online report: https://mokshtrehan.github.io/syde671-assignment2/

Contents

1.1 · Convolutions from scratch
1.2 · Finite difference operator
1.3 · Derivative of Gaussian
Gradient orientations
2.1 · Image sharpening
2.2 · Hybrid images
Color in hybrid images
2.3 · Gaussian and Laplacian stacks
2.4 · Multiresolution blending
Custom blends with irregular masks
Summary and observations
Reproduction and image credits

1.1 · Convolutions from scratch

Convolution method

Convolution flips the kernel in both dimensions and sums the products of overlapping samples. Both implementations use zero padding outside the image. The four-loop implementation iterates over image rows, image columns, kernel rows, and kernel columns. The two-loop implementation keeps the two kernel loops and operates on a complete shifted image slice at each step. Neither implementation uses a convolution routine, FFT, window-view helper, or image-processing library.

For a kernel of height Kh and width Kw, let K denote the kernel size along one axis. The padding is floor(K/2) samples before that axis and floor((K–1)/2) after it. For even kernels, this asymmetry matches SciPy's "same" output convention. Comparisons use non-symmetric odd and even kernels because a symmetric Gaussian alone cannot reveal a reversed kernel or a one-pixel offset.

Self-portrait results

The self portrait is converted to grayscale using 0.299R + 0.587G + 0.114B, with RGB values in [0, 1]. The working image is 384 × 512 pixels. A normalized 9 × 9 box filter and the derivative kernels D_x and D_Y are applied separately.

The box filter removes fine texture. D_x and D_Y emphasize changes across columns and rows, respectively; their signed responses share a symmetric color scale. Zero padding darkens the outer rim of the smoothed image and can produce derivative responses at the frame.

Runtime and numerical agreement

The runtime comparison uses a fixed 96 × 96 image resized from the self portrait. Each method uses float64 values, zero padding, and "same" output. The table reports the median of three calls after one warm-up, with a maximum absolute error of 8.88e-16 across all comparisons.

For an H × W image and a Kh × Kw kernel, both manual methods require O(HWKhKw) arithmetic. Operating on image slices removes Python's per-pixel loop overhead. The reported times describe performance on this computer.

Boundary conditions

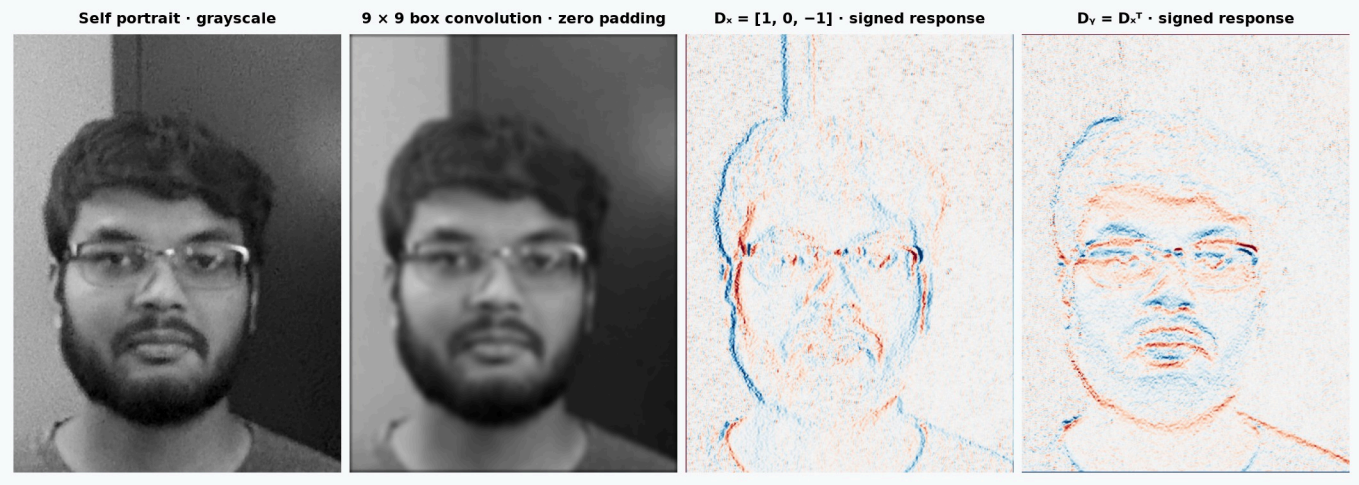
The boundary comparison uses a constant 25 × 25 image with value one and a normalized 9 × 9 box filter. Zero padding gives 25/81 at a corner and 5/9 at the midpoint of an edge; symmetric and periodic extension preserve one everywhere. The "same" setting controls output size, while the boundary setting supplies values outside the image.

Runtime and numerical agreement · 96 × 96 float64 input

Kernel	Method	Median time (ms)	Max. absolute error
3×5 asymmetric	Four loops	43.908	5.55e-16
3×5 asymmetric	Two loops	0.123	5.55e-16
3×5 asymmetric	SciPy convolve2d	0.209	0.00e+00
2×4 asymmetric (even)	Four loops	23.082	8.88e-16
2×4 asymmetric (even)	Two loops	0.08	8.88e-16
2×4 asymmetric (even)	SciPy convolve2d	0.138	0.00e+00
9×9 box	Four loops	209.111	7.77e-16
9×9 box	Two loops	0.609	7.77e-16
9×9 box	SciPy convolve2d	0.9	0.00e+00
1×3 derivative	Four loops	9.022	0.00e+00
1×3 derivative	Two loops	0.047	0.00e+00
1×3 derivative	SciPy convolve2d	0.095	0.00e+00

Boundary values for a constant image

SciPy boundary setting	Corner	Top edge midpoint	Center
fill	0.308642	0.555556	1.0
symm	1.0	1.0	1.0
wrap	1.0	1.0	1.0



Self portrait, box-filtered image, and signed horizontal and vertical derivatives. Red indicates positive responses and blue indicates negative responses. The shared derivative scale clips the strongest responses for display.

Input validation, kernel reversal, and padding

```
def _convolution_inputs(image, kernel):
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError("image and kernel must both be two dimensional")
    if not image.size or not kernel.size:
        raise ValueError("image and kernel must be nonempty")
    kh, kw = kernel.shape
    # Before = ceil((K-1)/2); after = floor((K-1)/2).
    padded = np.pad(image, ((kh // 2, (kh - 1) // 2),
                            (kw // 2, (kw - 1) // 2)), mode='constant')
    return image, kernel[::-1, ::-1], padded
```

Four-loop convolution

```
def convolve_four_loops(image, kernel):
    """True 2-D convolution: two spatial loops and two kernel loops."""
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for y in range(height):
        for x in range(width):
            for ky in range(kh):
                for kx in range(kw):
                    out[y, x] += flipped[ky, kx] * padded[y + ky, x + kx]
    return out
```

Two-loop convolution

```
def convolve_two_loops(image, kernel):
    """True 2-D convolution: kernel loops; spatial products are vectorized."""
    image, flipped, padded = _convolution_inputs(image, kernel)
    height, width = image.shape
    kh, kw = flipped.shape
    out = np.zeros_like(image)
    for ky in range(kh):
        for kx in range(kw):
            out += flipped[ky, kx] * padded[ky:ky + height, kx:kx + width]
    return out
```

1.2 · Finite difference operator

Gradient calculation

Let I be the grayscale image. Its horizontal and vertical derivative responses are $g_x = I * [1, 0, -1]$ and $g_y = I * [1, 0, -1]^T$, where $*$ denotes convolution. Reversing the kernel during convolution makes a left-to-right increase in intensity produce positive g_x. The gradient magnitude, $\sqrt{(g_x^2 + g_y^2)}$, combines the two responses into an unsigned measure of edge strength.

The supplied cameraman image has a thin white border. The crop (1, 7, 535, 541), given as (left, top, right, bottom) with the right and bottom coordinates excluded, removes that border. The remaining square is resized to 256 × 256 pixels with Lanczos interpolation. Calculations use floating-point intensities in [0, 1] without further normalization before thresholding.

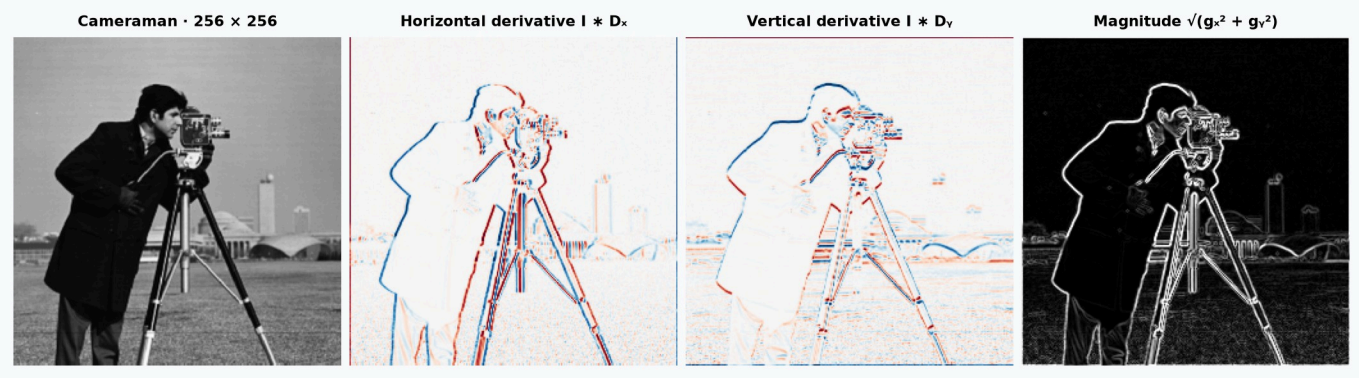
A pixel is classified as an edge when its gradient magnitude exceeds the threshold T. A seven-pixel frame is excluded from binary maps and edge-fraction measurements throughout Part 1. This margin excludes responses caused by zero padding and covers the largest Gaussian derivative support used below. The signed derivative and magnitude images show the complete responses.

Threshold comparison

Thresholds of 0.08, 0.16, and 0.24 retain 23.7%, 12.8%, and 9.0% of the evaluated interior, respectively. The selected threshold, 0.16, preserves the person, camera, and tripod more clearly than 0.24 while rejecting much of the ground texture retained at 0.08. Selection is based on this visual balance; detection accuracy is not measured against annotated edges.

Threshold sensitivity

Threshold	Interior edge fraction	Selection
0.08	23.68%	Comparison
0.16	12.81%	Selected
0.24	8.99%	Comparison



Cameraman finite differences and gradient magnitude. Signed derivatives share the range [−0.65, 0.65]; magnitude uses [0, 0.65].



Threshold comparison for the finite-difference magnitude. The selected threshold is 0.16. The black outer frame is excluded from evaluation.

Gradient and edge magnitude

```
gx = convolve_two_loops(image, DX)
gy = convolve_two_loops(image, DY)
magnitude = np.sqrt(gx * gx + gy * gy)
edges = magnitude > threshold
```

1.3 · Derivative of Gaussian

Gaussian derivative kernels

A 13 × 13 Gaussian kernel G with σ = 2 pixels is formed from the outer product of cv2.getGaussianKernel(13, 2) with itself. The image is first smoothed with G, then differentiated with D_x and D_Y.

Alternatively, Kx = G ∗ D_x and Ky = G ∗ D_Y combine smoothing and differentiation into one image convolution per direction. These finite-difference derivatives of the sampled Gaussian have supports of 13 × 15 and 15 × 13 pixels because full kernel convolution retains the extra two samples.

Boundary conditions

Convolution associativity gives I ∗ (G ∗ D) = (I ∗ G) ∗ D for an image extended with zeros. Applying “same” output after each stage crops the intermediate blurred image and removes values beyond the original frame. These removed values affect the subsequent derivative near the boundary.

The two successive “same” convolutions differ from a single Gaussian derivative convolution by at most 0.307 over the complete image, while the evaluated interior agrees to 8.95e-16. Retaining the full blurred image, applying the derivative, and cropping once reduces the maximum difference to 8.95e-16. The difference images show where the intermediate crop affects the result.

Edge comparison

The derivative of Gaussian (DoG) suppresses fine grass texture and produces smoother, wider contours than finite differences on the original image. Smoothing can also merge nearby edges and remove weak detail. The selected thresholds are T = 0.08 for DoG and T = 0.16 for the original gradients because smoothing reduces response amplitudes. These thresholds support a visual comparison at separately chosen settings; they do not establish a difference in detection accuracy.

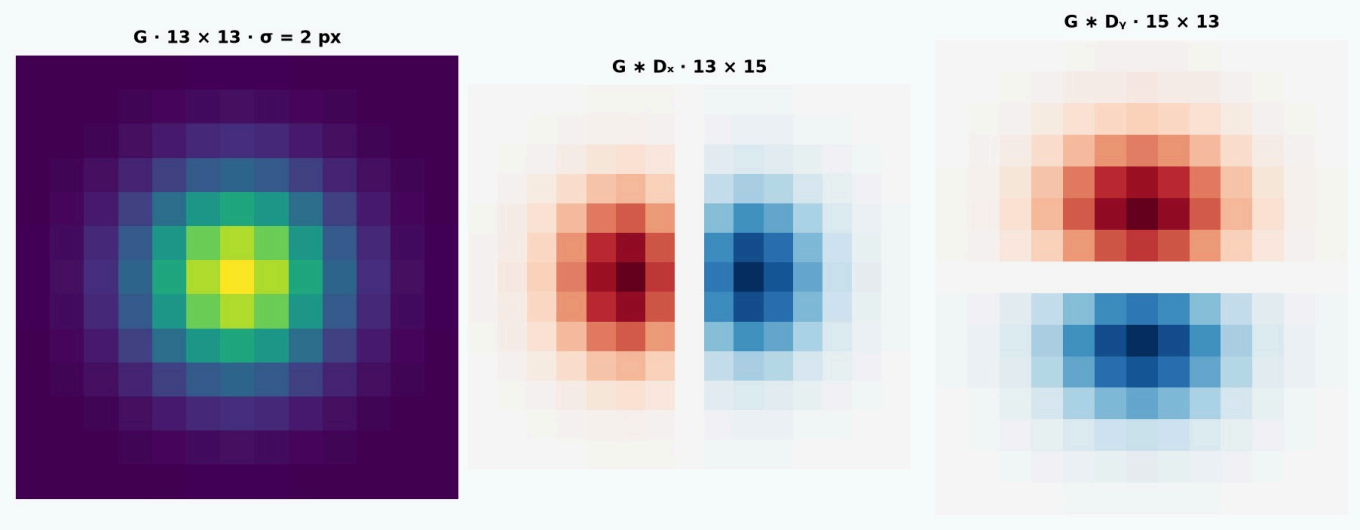
Noise sensitivity

The noise experiment adds Gaussian noise with σ = 0.08 using a fixed random seed, then clips intensities to [0, 1]. Each noisy gradient is compared with the same method applied to the clean image. This comparison measures the change caused by noise while retaining each method’s smoothing effect.

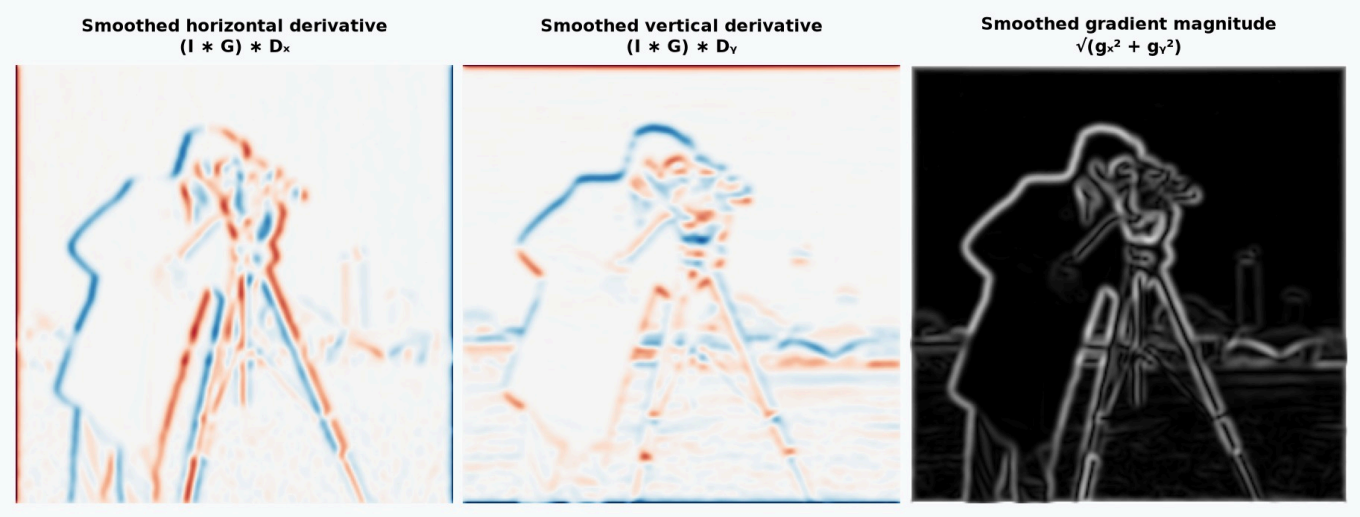
The root mean square (RMS) of the interior vector-gradient residual decreases from 0.15226 for finite differences to 0.01030 for DoG, a factor of 14.8. Gaussian smoothing therefore reduces sensitivity to the added noise in this experiment. The measurement describes this image and noise realization; it does not measure recovery of detail removed by smoothing.

Convolution agreement and noise sensitivity

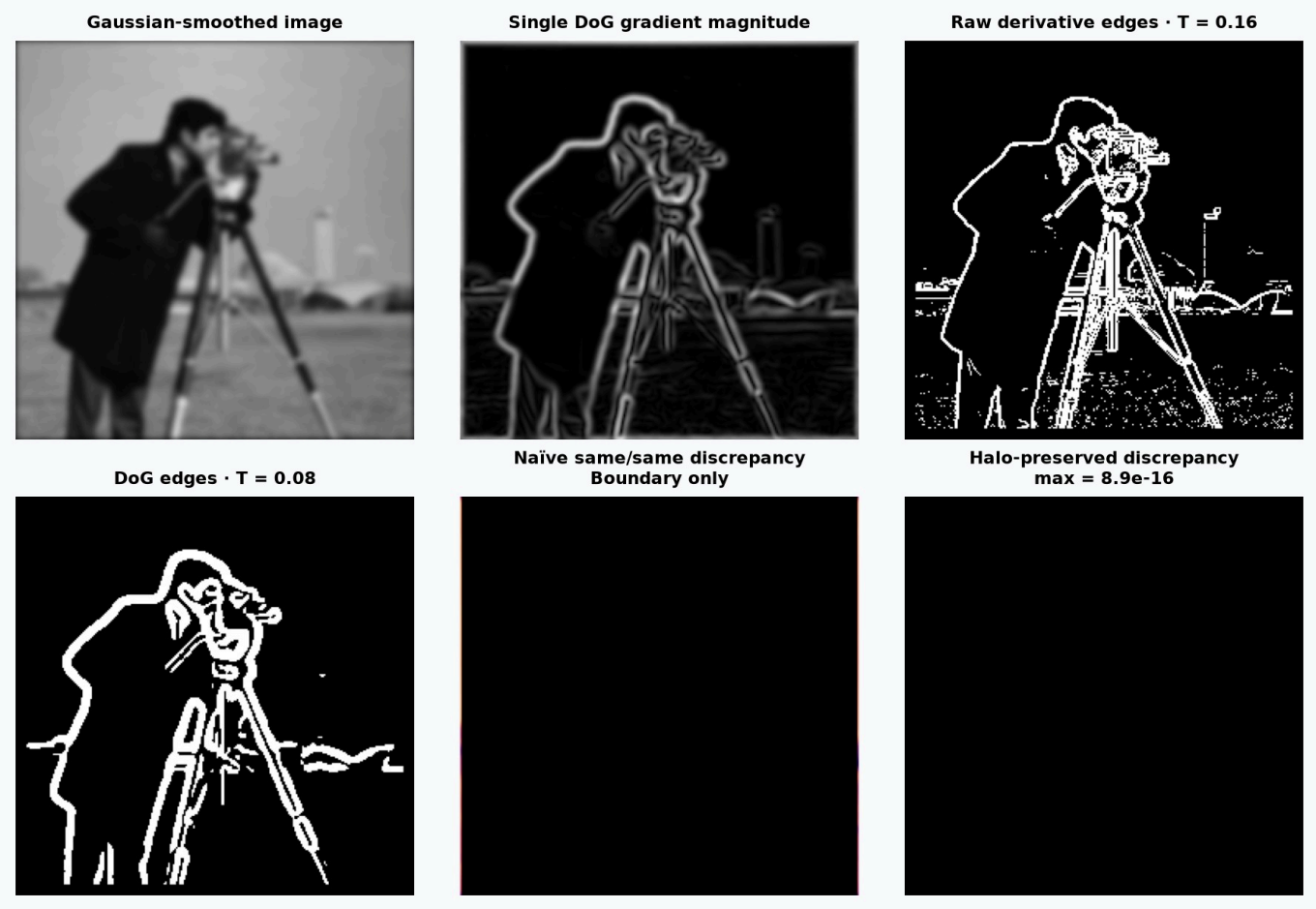
Measurement	Value
Two successive “same” convolutions vs. DoG · maximum error	0.307441
Two successive “same” convolutions vs. DoG · interior maximum error	8.95e-16
Full intermediate image vs. DoG · maximum error	8.95e-16
Finite-difference gradient residual RMS	0.152265
DoG gradient residual RMS	0.010304



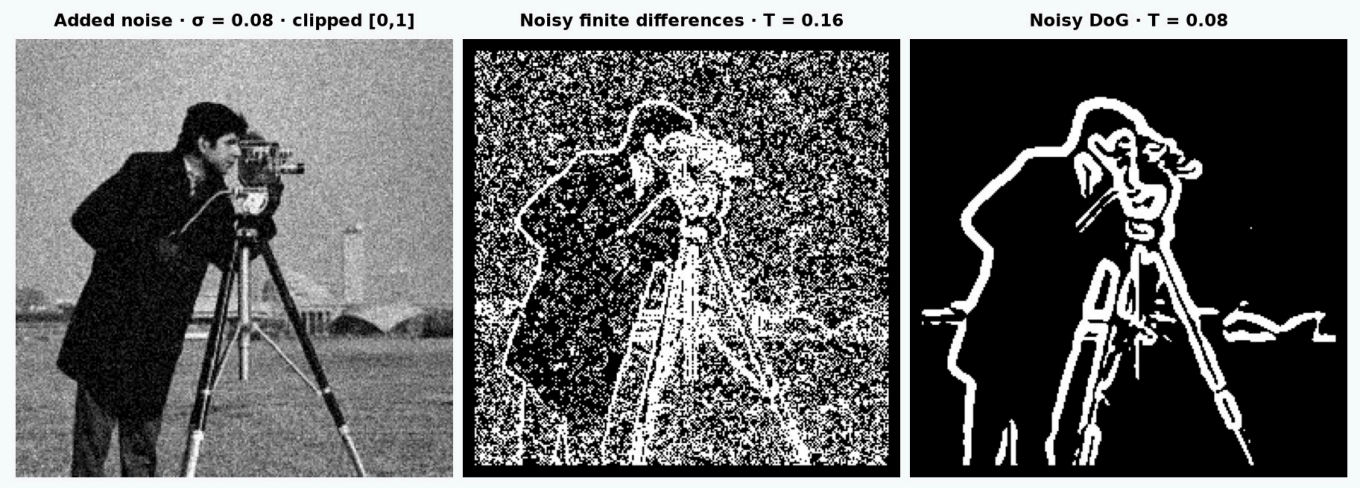
Gaussian kernel and its sampled horizontal and vertical derivatives. The derivative kernels share a signed scale; the Gaussian uses a separate nonnegative scale.



Finite differences after Gaussian smoothing, using two successive “same” convolutions with zero padding. Signed derivatives share [−0.35, 0.35]; magnitude uses [0, 0.35].



Gaussian derivative results, thresholded edges, and boundary differences. Retaining the complete intermediate blurred image restores agreement between sequential and combined convolution.



Cameraman image with added Gaussian noise and the resulting edge maps. Gaussian smoothing suppresses many of the noise responses visible with finite differences.

Build Gaussian derivative kernels

```
g1 = cv2.getGaussianKernel(13, 2.0)
G = g1 @ g1.T
Kx = convolve2d(G, DX, mode="full")
Ky = convolve2d(G, DY, mode="full")
gx = convolve2d(image, Kx, mode="same", boundary="fill")
gy = convolve2d(image, Ky, mode="same", boundary="fill")
```

Gradient orientations

Orientation calculation

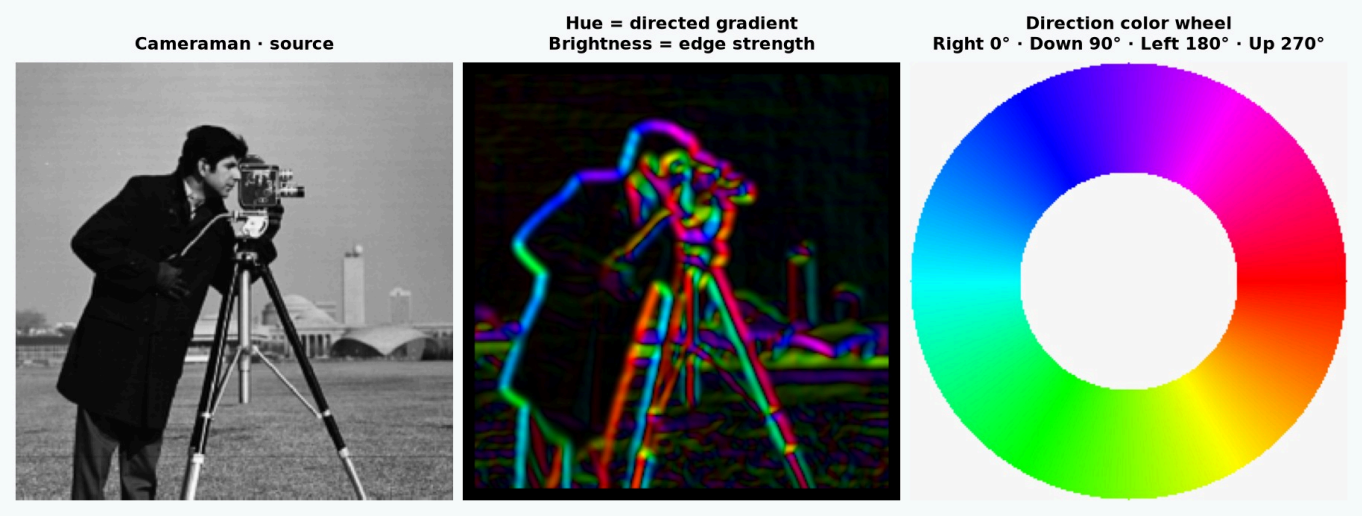
Gradient orientation uses the Gaussian-derivative responses gx and gy from Section 1.3. The calculation uses arithmetic operations without calling an arctangent, atan2, or angle routine. For nonzero gradients, r = min(|gy|, |gx|)/max(|gy|, |gx|) lies between zero and one. For r > $\sqrt{2}-1$, the identity atan(r) = π/4 + atan((r−1)/(r+1)) reduces the magnitude of the series argument to at most $\sqrt{2}-1$.

The arctangent is approximated with 18 terms of $z - z^3/3 + z^5/5 - \ldots$, where z is the reduced argument. Coordinate swaps and the signs of the gradient components determine the quadrant. A zero gradient is assigned orientation zero and displayed black.

Color representation and validation

The HSV display maps orientation θ to hue (θ mod 2π)/(2π), with saturation one. Brightness is min(magnitude / p99.5, 1)^0.7, where p99.5 = 0.2386 is the 99.5th percentile of gradient magnitude in the evaluated interior. Stronger gradients appear brighter, while weak background responses remain subdued. The seven-pixel boundary margin is displayed black.

Color indicates the direction of increasing intensity, perpendicular to the edge tangent. Image rows increase downward, so 90° points down. Comparison with NumPy atan2, used only for validation, gives a maximum angular error of 4.44e-16 radians for this image.



Gradient orientation and direction color wheel. Hue represents direction in image coordinates; brightness represents edge strength. The seven-pixel boundary margin is excluded.

Arithmetic-only orientation implementation

```
def orientation_without_angles(gx, gy, terms=18):
    gx, gy = np.broadcast_arrays(np.asarray(gx, float), np.asarray(gy, float))
    if not np.all(np.isfinite(gx)) or not np.all(np.isfinite(gy)):
        raise ValueError('gradient components must be finite')
    if terms < 1:
        raise ValueError('terms must be positive')
    ax, ay = (np.abs(gx), np.abs(gy))
    high, low = (np.maximum(ax, ay), np.minimum(ax, ay))
    ratio = np.divide(low, high, out=np.zeros_like(high), where=high != 0)
    reduce = ratio > np.sqrt(2.0) - 1.0
    z = np.where(reduce, (ratio - 1.0) / (ratio + 1.0), ratio)
    z2 = z * z
    poly = np.zeros_like(z)
    for n in range(terms - 1, -1, -1):
        poly = (-1.0) ** n / (2 * n + 1) + z2 * poly
    small_angle = z * poly + np.where(reduce, np.pi / 4, 0.0)
    angle = np.where(ay > ax, np.pi / 2 - small_angle, small_angle)
    angle = np.where(gx < 0, np.pi - angle, angle)
    angle = np.where(gy < 0, -angle, angle)
    return np.where(high == 0, 0.0, angle)
```

2.1 · Image sharpening

Unsharp masking

For an input image I and Gaussian kernel G, let B = G ∗ I be the low-pass image and H = I − B its high-frequency residual. Unsharp masking adds a scaled residual: S = I + αH, where α controls the sharpening amount.

Expanding the residual gives S = (1 + α)I − α(G ∗ I). The equivalent convolution kernel is K = (1 + α)δ − αG, where δ is the unit impulse, so S = K ∗ I requires one convolution. K sums to one and preserves a constant image when the boundary extension also preserves constants.

The experiment uses a 13 × 13 Gaussian with σ = 2 pixels and symmetric boundary extension. Each RGB channel is processed independently in float64, retaining signed residuals and values outside [0, 1] until display export. Symmetric extension avoids the dark borders introduced by zero padding. The Taj Mahal and an original street photograph are processed at α = 0.5, 1, and 2.

Sharpening results

Increasing α strengthens existing edges and texture, but also amplifies noise and halos and increases values outside the display range. Residuals are displayed as 0.5 + 2H, with middle gray representing zero and a fixed amplification of two. No independent normalization is applied to these residuals. The table reports the fraction of scalar RGB values outside [0, 1] before display clipping.

The single-convolution result agrees with S = I + α(I − G ∗ I) at α = 1 to a maximum absolute error of 1.78e-15 across the two images. This comparison checks the equivalence of the residual and kernel formulations.

Recovery after blurring

The recovery experiment first blurs the original street image, then sharpens that blurred image with the same Gaussian at each tested α. Mean squared error (MSE) is measured over RGB values relative to the original image, both before and after display clipping. For the reference range [0, 1], peak signal-to-noise ratio is PSNR = 10 log10(1/MSE), in decibels.

Among the tested settings, α = 2 gives the lowest clipped RGB MSE. The MSE changes from 0.001660 after blurring to 0.001099 after sharpening, while PSNR changes from 27.80 to 29.59 dB. This parameter is selected against the original reference and applies to this comparison.

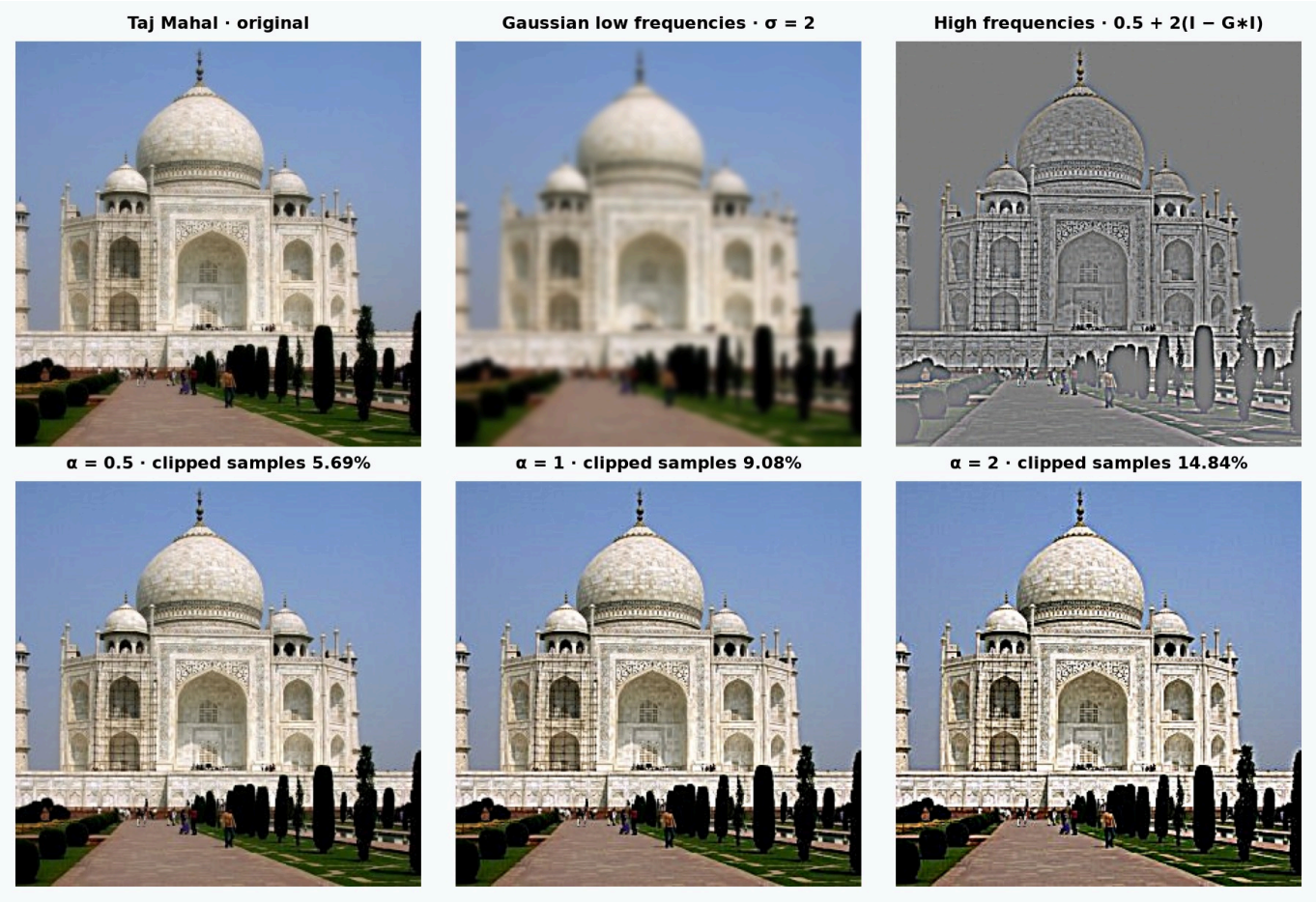
Sharpening makes blurred transitions steeper but leaves differences from the original image. If G(f) is the Gaussian frequency response, the combined blur and sharpening response is G(f)[1 + α − αG(f)], which generally differs from one. Frequencies strongly attenuated by the blur remain attenuated after this finite amplification, and clipping can remove additional information. The enlarged crop, residual map, and nonzero MSE show that increased edge contrast does not restore all original detail.

RGB values outside the display range

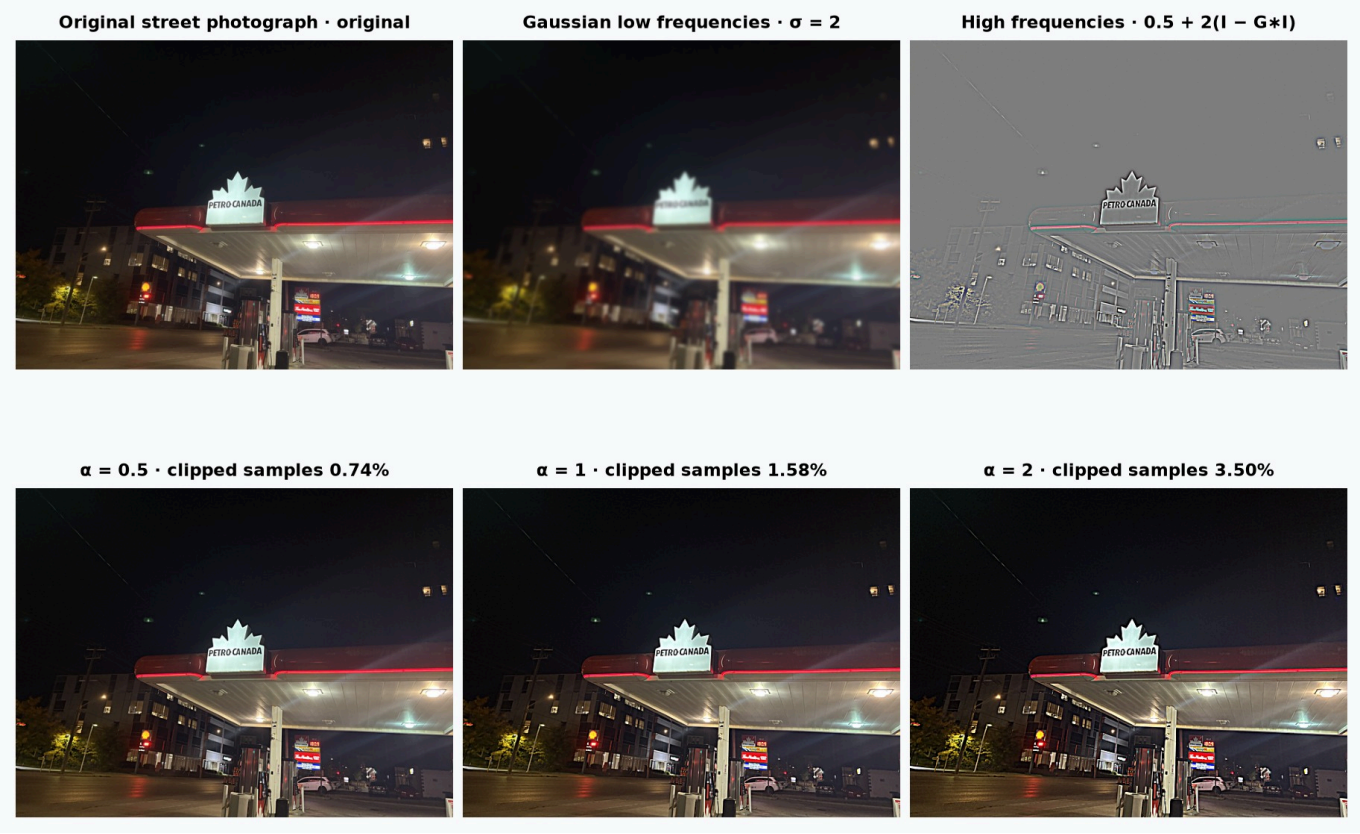
Image	α	Values outside [0, 1]
Taj Mahal	0.5	5.687%
Taj Mahal	1.0	9.075%
Taj Mahal	2.0	14.845%
Original street photograph	0.5	0.741%
Original street photograph	1.0	1.581%
Original street photograph	2.0	3.504%

Recovery relative to the original street image · RGB reference in [0, 1]

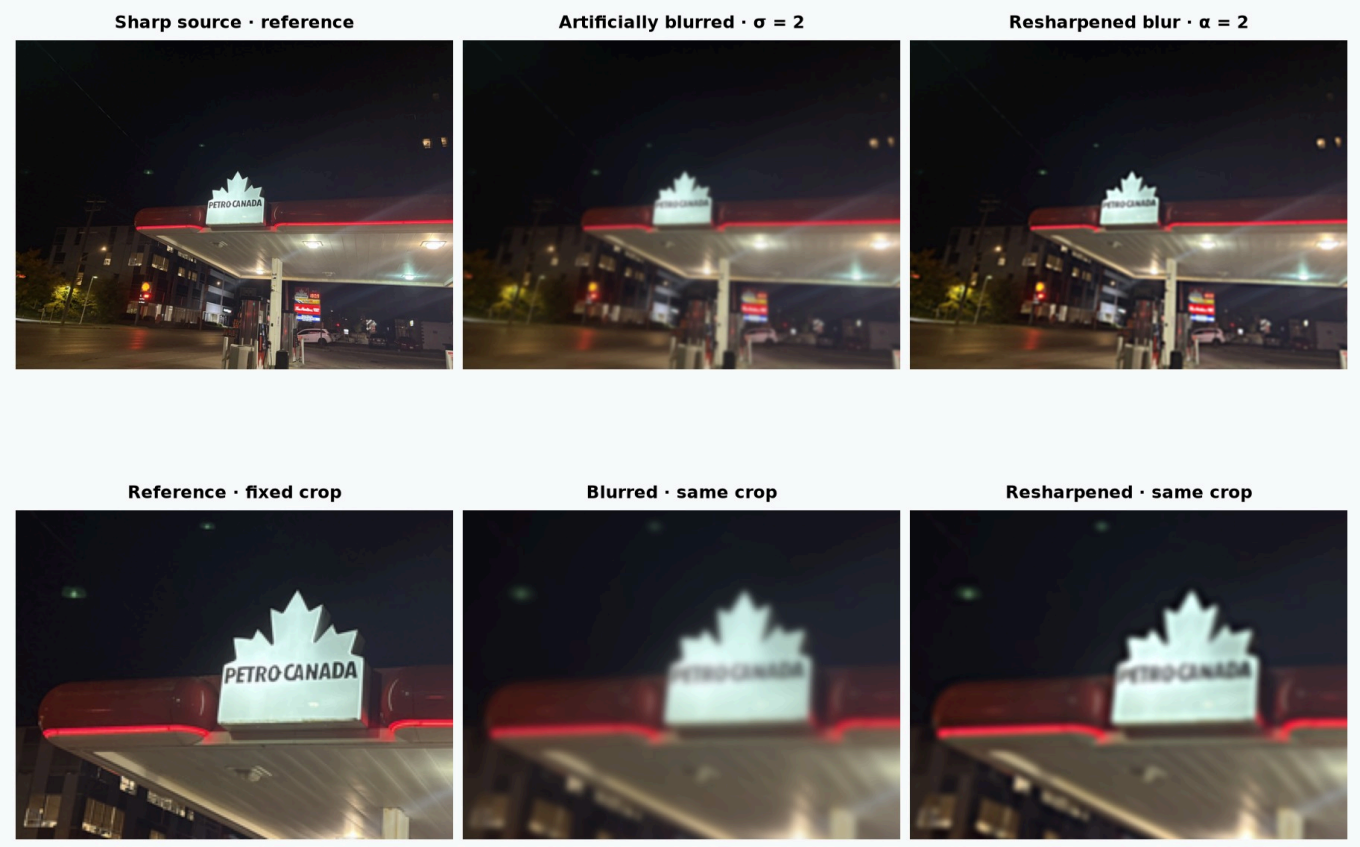
Stage	α	Unclipped MSE	Unclipped PSNR (dB)	Clipped MSE	Clipped PSNR (dB)
Original	—	0	∞	0	∞
Blurred	—	0.001660	27.80	0.001660	27.80
Resharpened	0.5	0.001429	28.45	0.001429	28.45
Resharpened	1.0	0.001259	29.00	0.001259	29.00
Resharpened	2.0	0.001101	29.58	0.001099	29.59



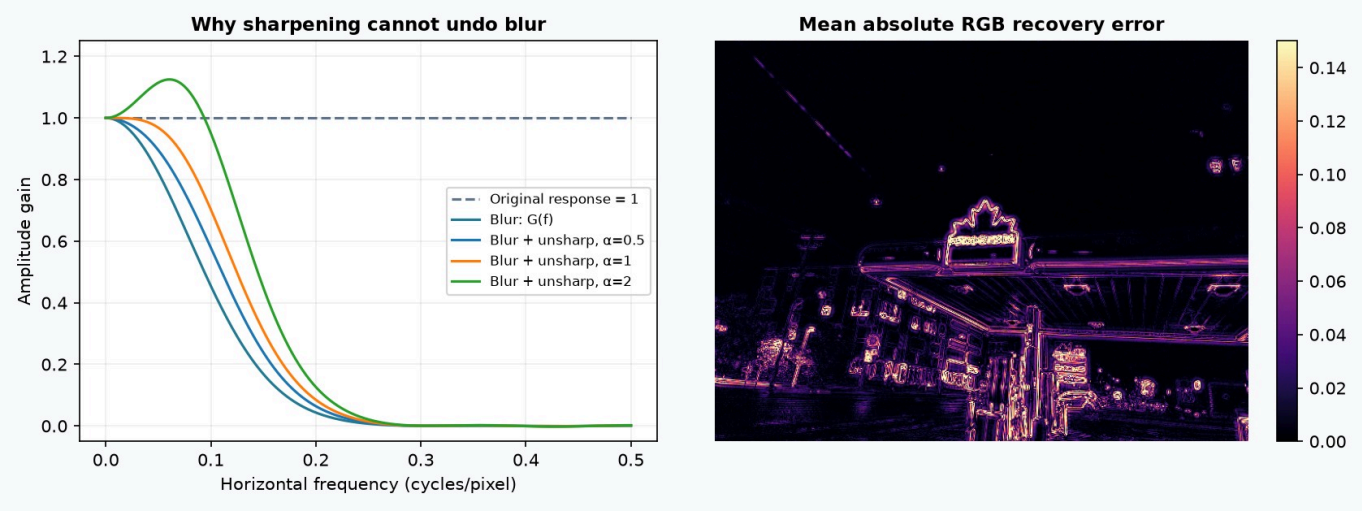
Taj Mahal: original image, Gaussian blur, high-frequency residual, and sharpening at α = 0.5, 1, and 2. The residual is displayed as 0.5 + 2H. Final display values are clipped to [0, 1].



Original street photograph: original image, Gaussian blur, high-frequency residual, and sharpening at α = 0.5, 1, and 2. The residual is displayed as 0.5 + 2H. Final display values are clipped to [0, 1].



Blur and sharpening of the original street photograph. The lower row shows the same fixed region from each image. α = 2 gives the lowest clipped RGB MSE among the three tested settings.



Left: horizontal frequency response of the discrete Gaussian and the combined blur and sharpening operations. Right: mean absolute RGB error against the sharp reference at the selected setting.

Single-convolution sharpening core (each RGB channel)

```
K = -alpha * G
K[G.shape[0] // 2, G.shape[1] // 2] += 1 + alpha
for c in range(3):
    sharp[..., c] = convolve2d(image[..., c], K, mode="same", boundary="symm")
```

2.2 · Hybrid images

Hybrid construction

A hybrid image combines a blurred image with fine detail from a second, aligned image: hybrid = G_sigmaL * A + gain (B - G_sigmaH * B). G_sigma is a Gaussian kernel with standard deviation sigma, and * denotes convolution. A is the human portrait and B is the cat photograph. The three examples are Derek + Nutmeg and two custom portrait/cat pairs.

Image registration

The initial alignment maps both pairs of eye centers to (170,210) and (330,210). Each two-point similarity transform combines rotation, uniform scale and translation. Exact eye alignment leaves the human mouth below the cat's muzzle in the Derek and astronaut pairs.

For these two pairs, the human image is scaled by 0.85 about the common eye midpoint (250,210) and shifted upward by 8 pixels. Its eye positions become (182,202) and (318,202), while the cat positions remain fixed. This reduces the extra mouth below the muzzle, but the pupils no longer coincide exactly. Obama retains the common eye positions.

Each original is resampled once with the composed similarity transform, using bilinear interpolation and replicated borders. The face proportions are unchanged. The original landmarks and exact 2 x 3 transformation matrices are saved with the experiment records.

Filter settings

A high-pass sigma of 1.5 pixels removes more broad cat shading than sigma 5, so less cat structure remains in the reduced views. The selected low-pass sigma is 8 pixels for Derek and the astronaut, and 6 pixels for Obama. Their respective gains are 2.25, 2.7 and 2.2. These gains retain visible fur detail.

The six filter-width and gain trials keep the selected alignment fixed. Increasing the low-pass sigma to 14 blurs the astronaut's eyes and smile in the reduced view. Increasing the high-pass sigma from 1.5 to 3 retains broader cat structure, which competes with the human face. The supplementary comparisons vary alignment and filtering separately. These settings are selected visually for these particular inputs.

Appearance at different scales

The selected astronaut hybrid emphasizes the cat's fur and eye outlines at 512 pixels. At 64 pixels, the astronaut's eyes and smile become clearer. The 128- and 256-pixel views retain recognizable features from both sources.

The refined human alignment reduces the extra smile below the cat's muzzle. A 10% gain reduction from the reference setting limits clipping while retaining fine detail. The final clipped fraction is still slightly higher than for the larger human crop with exactly aligned eyes. The comparison therefore remains a partial change in appearance with scale.

To inspect the effect, view the largest image at normal size and then move farther away, keeping its display size fixed. The reduced versions provide a second comparison. The apparent change depends on screen size, zoom and eyesight, so the pixel-size sequence does not establish a calibrated viewing distance.

Frequency analysis

A Gaussian has no sharp frequency cutoff. Its approximate half-amplitude frequency is sqrt(2 ln 2) / (2 pi sigma) cycles per output pixel. For the selected astronaut pair, this is 0.0234 for the low pass and 0.1249 for the complementary high pass. Filtering uses spatial convolution; the Fourier plots show the resulting frequency content.

Each Fourier spectrum displays log(1 + 1000000 |fftshift(fft2(luminance))| / N), where N is the number of image pixels. Division by N permits comparison of differently sized inputs, and both figures use one shared display maximum. No window or mean subtraction is applied before the Fourier transform.

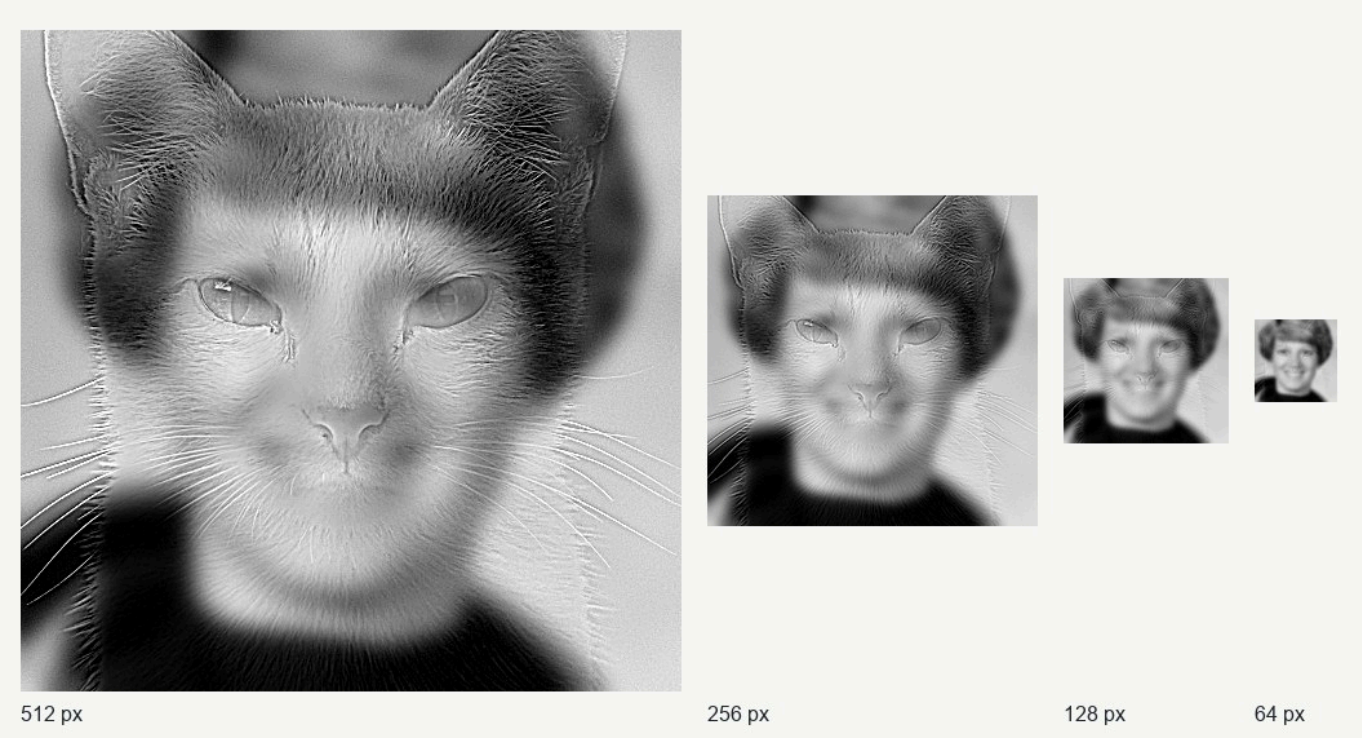
Numerical representation

Filtering uses reflected boundaries, and filtering and addition retain signed float64 values. Intermediate bands are neither clipped nor contrast-normalized. Display exports clamp values to [0,1] and quantize them to 8 bits; the table reports the fraction of scalar channel values clipped. Supplementary figures include the remaining alignment, component and parameter comparisons, together with the unsuccessful Earth + Moon hybrid.

Selected settings at 512 x 512 pixels

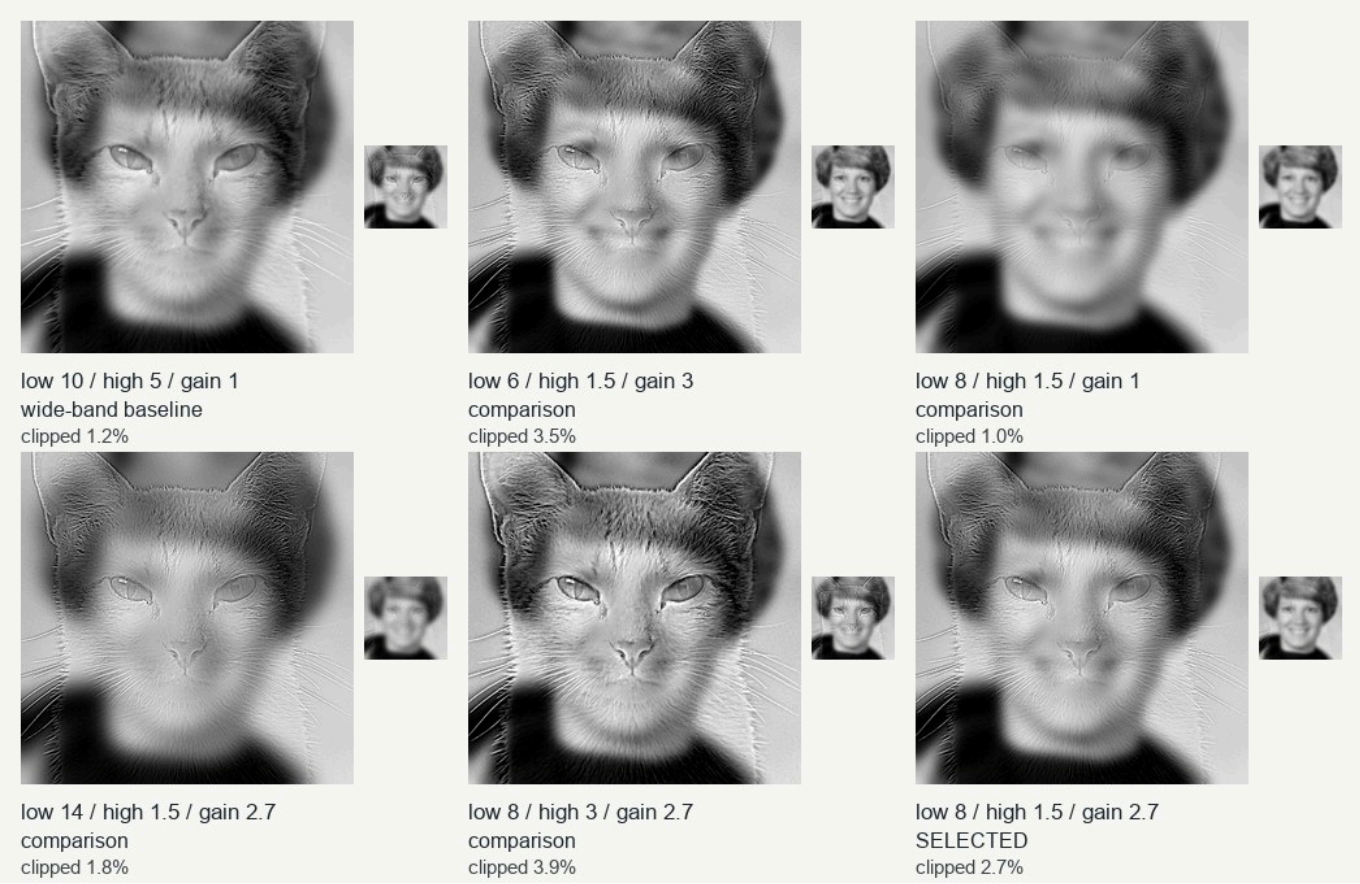
Pair	Low-pass sigma (px)	High-pass sigma (px)	Gain	Values clipped
Derek + Nutmeg	8	1.5	2.25	2.36%
Astronaut + portrait cat	8	1.5	2.7	2.74%
Obama + tabby	6	1.5	2.2	2.07%

Hybrid appearance at different display sizes



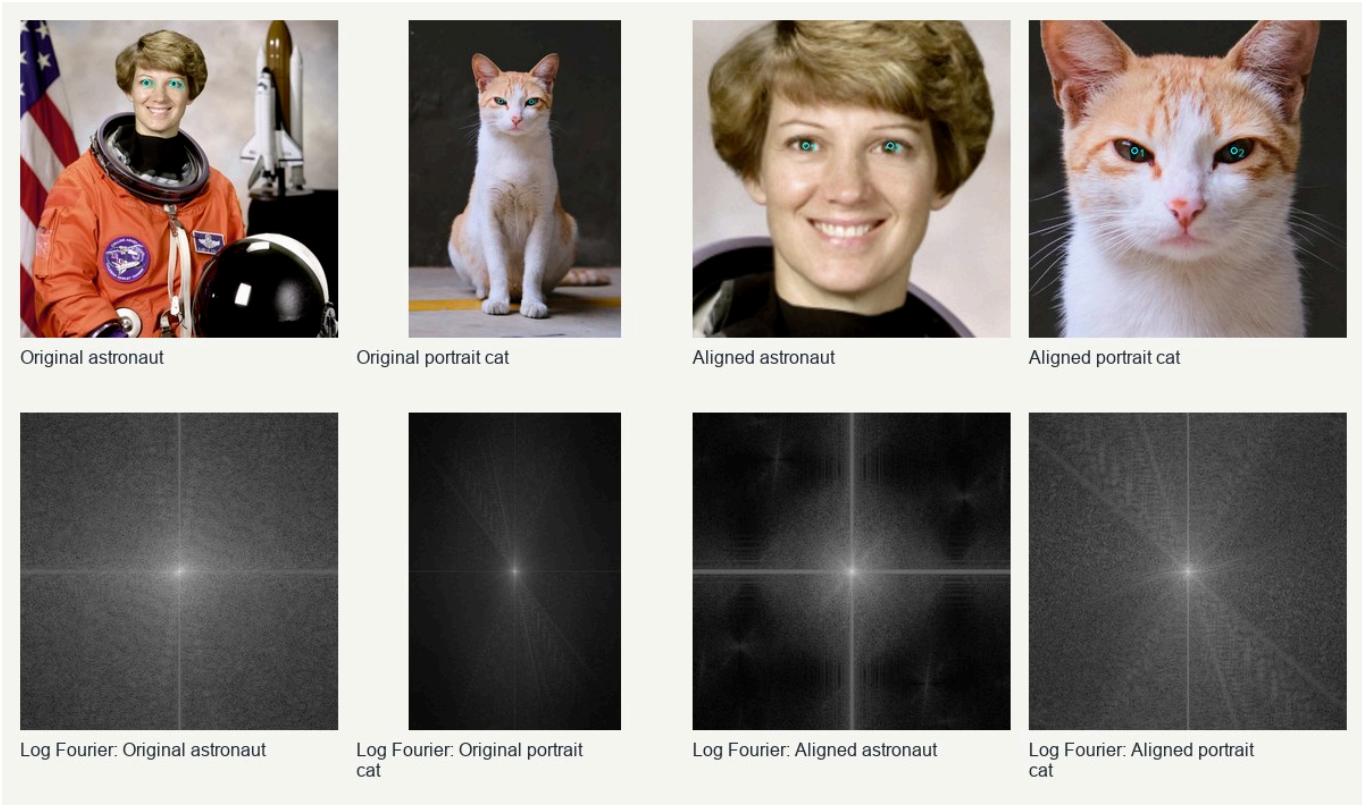
Astronaut + portrait cat: the same hybrid at 512, 256, 128 and 64 pixels, reduced with Lanczos. Cat fur and eye outlines are more visible at the largest size; the astronaut's eyes and smile become clearer at smaller sizes. Both sources remain visible at intermediate sizes.

Filter widths and detail gain



Astronaut + portrait cat: filter-width and gain trials, synthesized at 512 x 512 and displayed at 256 and 64 pixels. The selected panel uses the final settings. Alignment remains fixed across the trials.

Original sources, alignment and Fourier spectra



Original and aligned astronaut and portrait-cat inputs. Cyan circles show the alignment landmarks; spectra are calculated from the unmarked images. Both Fourier figures use the same logarithmic scale and display maximum.

Filtered components and reconstruction



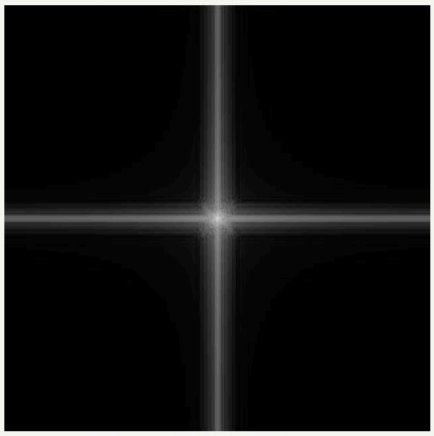
Gaussian low pass



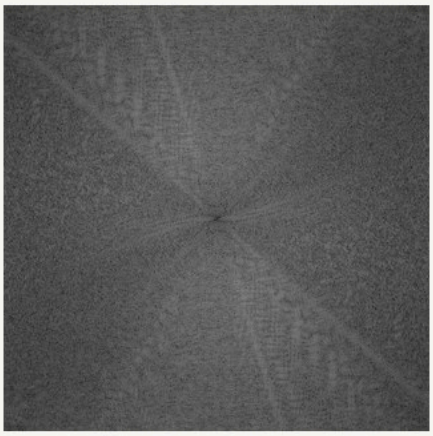
Gain-scaled signed high pass



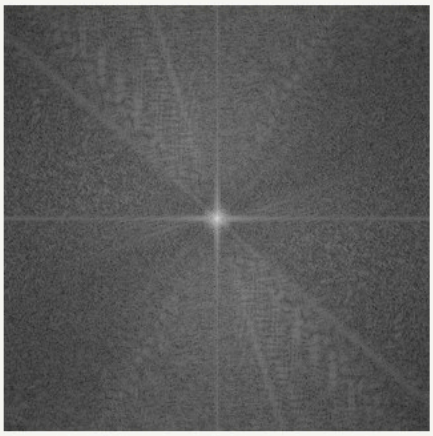
Final hybrid



Log Fourier: Gaussian low pass



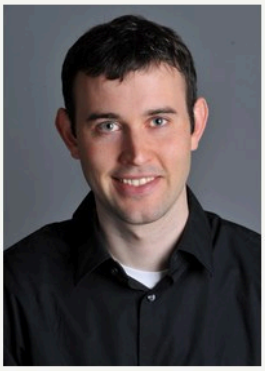
Log Fourier: Gain-scaled signed high pass



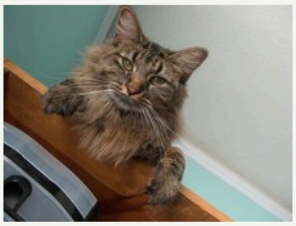
Log Fourier: Final hybrid

The low-pass spectrum is concentrated near the center. The high-pass component suppresses central energy and retains fine detail. Adding the two components produces the hybrid; the signed high-pass component is displayed as 0.5 + gain H. Both Fourier figures use the same logarithmic scale and display maximum.

Derek + Nutmeg



Original low source



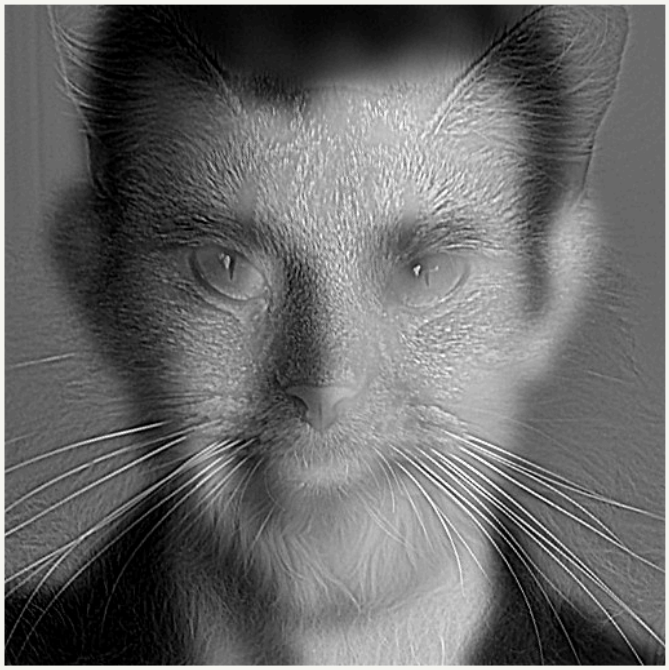
Original high source



128 px



64 px



Hybrid / 512 px

Derek + Nutmeg: original sources and the final hybrid at 512, 128 and 64 pixels. The adjusted human scale and position reduce the additional mouth visible below the cat muzzle. Cat fur and eye outlines remain visible at 512 pixels. Derek's facial structure becomes clearer at 64 pixels; intermediate sizes retain features from both sources.

Obama + tabby



Original low source



Original high source



128 px



64 px



Hybrid / 512 px

Obama + tabby: original sources and the final hybrid at 512, 128 and 64 pixels. Gain 2.2 retains the tabby's fine fur texture. Higher gain clips more bright fur pixels. Obama's smile becomes clearer at reduced size, while whiskers remain visible at intermediate sizes.

Hybrid synthesis

```
low = gaussian(A, sigma_low)
high = B − gaussian(B, sigma_high)
hybrid = low + gain ∗ high
```

Additional parameter comparisons are available in the online supplement.

Color in hybrid images

Four variants retain color in neither source, only the low-frequency source, only the high-frequency source, or both. Color in the low-frequency source preserves the astronaut's broad skin and hair regions. Color in the high-frequency source appears around fine cat edges, but the filtering removes the broad coat color.

For this pair, color in the low-frequency source agrees with the face visible at smaller sizes, while grayscale fur remains visible at larger sizes. Retaining color in both sources adds competing hues around some cat edges. The main examples use grayscale so that the change in visible subject can be assessed without this broad color difference.

Color by frequency band



Four color variants with identical alignment, filter widths and gain. Luminance conversion uses weights 0.2126, 0.7152 and 0.0722. Color in the low-frequency source preserves broad skin and hair regions; color in the high-frequency source remains mainly around cat edges and fur.

2.3 · Gaussian and Laplacian stacks

Input preparation

The supplied apple and orange images are burt_apple.png and burt_orange.png. Each 740 × 530 input is resized once to a common working size of 512 × 367 pixels using area interpolation. This reduces the computational cost while preserving the original aspect ratio to the nearest output pixel.

Gaussian and Laplacian decomposition

A Gaussian stack retains the same image dimensions at every level. G0 = I is the original image; G1 through G5 are formed by independently blurring I at σ = 2, 4, 8, 16, and 32 pixels. Each σ therefore describes the smoothing relative to the original image. The stack is constructed explicitly without downsampling or a built-in pyramid routine.

The Laplacian detail bands are Li = Gi – G(i+1) for i = 0, ..., 4, and L5 = G5 is the coarse residual. Summing the detail bands gives I – G5; including the residual restores the large-scale brightness and color. The complete sum reconstructs I up to floating-point rounding. The table reports the reconstruction error for each input and for the blended image.

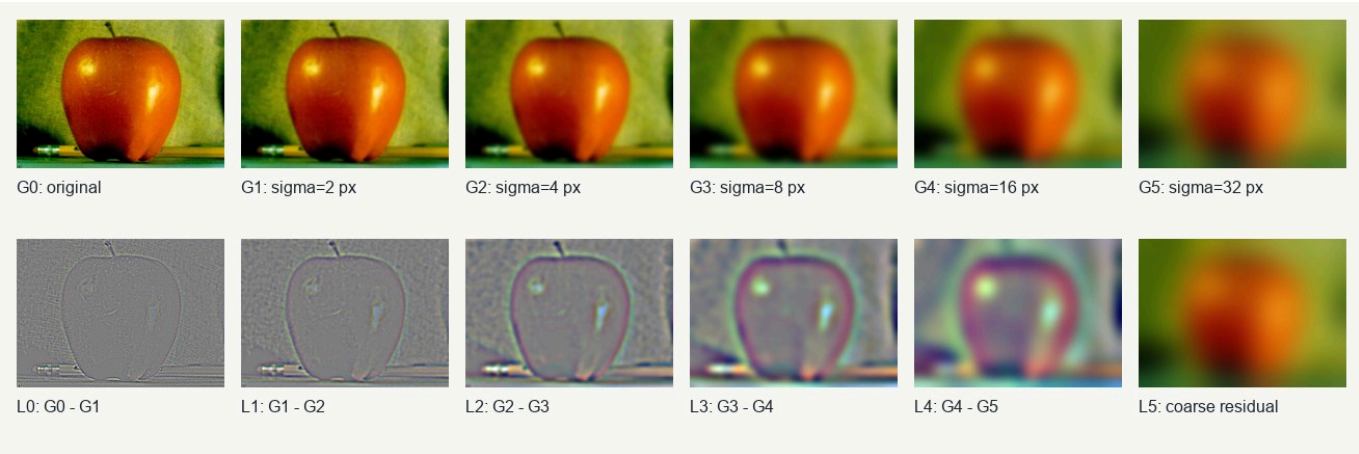
Mask and reconstruction

A binary mask selects apple on the left and orange on the right. Its Gaussian stack uses the same smoothing scales as the image stacks. At level i, Mi is the mask and LAi and LBi are the apple and orange Laplacian bands; the combined band is Si = Mi LAi + (1 – Mi) LBi. This expression also applies to the coarse residual, and summing all Si gives the final image.

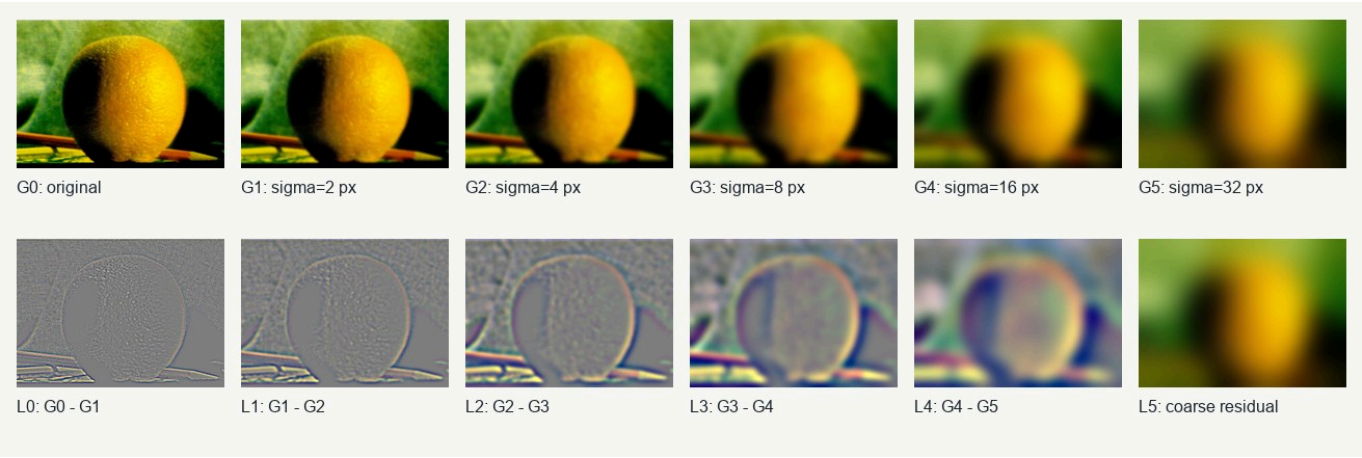
The mask has a sharp transition at the finest level and progressively wider transitions at coarser levels. Fine texture and broad changes in brightness and color are therefore blended over different spatial ranges. The contribution figure shows selected detail levels and the complete reconstruction.

Reconstruction accuracy

Comparison	Maximum absolute error
Apple reconstruction	1.110e-16
Orange reconstruction	2.220e-16
Final image vs. sum of mixed bands	3.331e-16



Apple: Gaussian stack above and Laplacian stack below, including the coarse residual. Every level retains the working resolution of 512 × 367 pixels. Signed detail bands use 0.5 + 2.5L for display; the residual uses its actual intensity.



Orange: Gaussian stack above and Laplacian stack below, including the coarse residual. Every level retains the working resolution of 512 × 367 pixels. Signed detail bands use 0.5 + 2.5L for display; the residual uses its actual intensity.



Gaussian stack of the left/right selection mask, using the same scales as the image stacks. The transition widens at coarser levels. White selects apple and black selects orange.

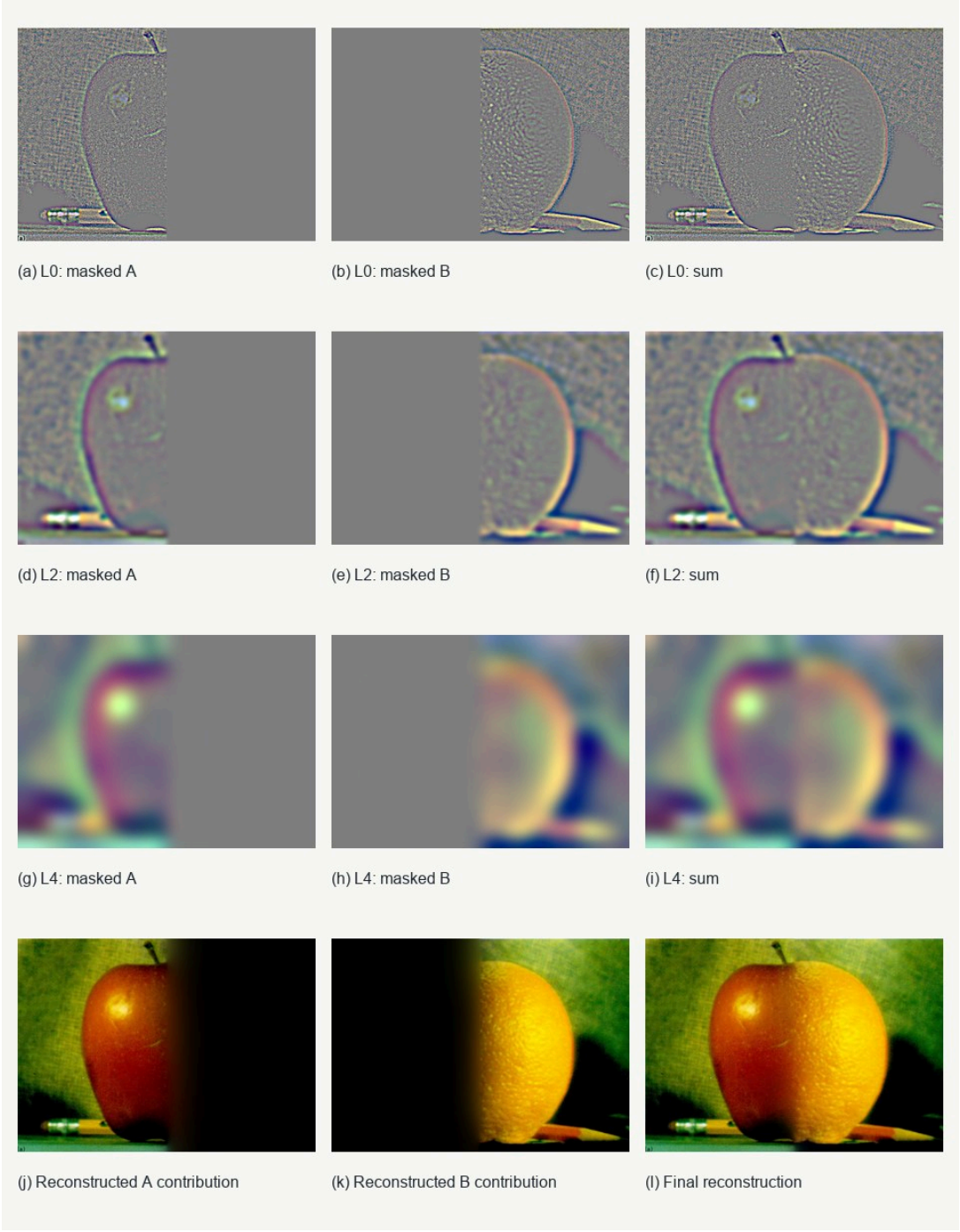


Figure 3.42 (a)–(l) layout: rows 1–3 show levels 0, 2, and 4; columns show the masked apple, masked orange, and their sum. Row 4 sums all five detail bands and the coarse residual to reconstruct the two contributions and final oraple. Detail bands share a symmetric display scale within each row; reconstructed images use actual [0, 1] intensity.

Stack construction and reconstruction

```
G = [I] + [gaussian(I, 2 * 2**i) for i in range(5)]
L = [G[i] - G[i+1] for i in range(5)] + [G[-1]]
S = [M[i] * LA[i] + (1-M[i]) * LB[i] for i in range(6)]
result = sum(S)
```

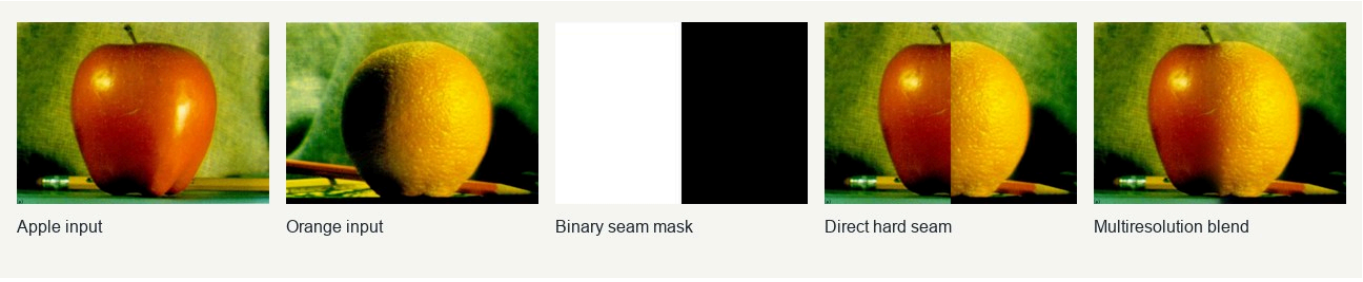
2.4 · Multiresolution blending

Comparison and results

The apple and orange are combined using the same inputs and initial selection mask for both methods. A direct binary cut produces a visible vertical change in brightness and color. Multiresolution blending combines corresponding Laplacian bands with Gaussian mask weights, spreading the transition according to scale while retaining local texture.

The mean absolute RGB difference between the columns adjoining the seam decreases from 0.1716 for the binary cut to 0.0279 for the multiresolution blend. This local measurement is consistent with the smoother visible transition. It describes the seam discontinuity rather than overall perceptual quality.

The seam measurement uses the floating-point reconstruction before clipping. Values remain unclipped throughout stack construction and reconstruction, then are limited to [0, 1] for display export. This affects 2.35% of scalar RGB values.



Supplied inputs, selection mask, direct binary cut, and multiresolution oraple. Both methods use the same 512 × 367 images with preserved aspect ratio. The seam lies between zero-based columns 255 and 256.

Custom blends with irregular masks

Image pairs and registration

The cat composite places the tabby's central face within the orange-and-white cat's ears, outer whiskers and chest. A 24-vertex polygon follows the crown, temples, cheeks and chin, keeping the forehead stripes inside the selection. A similarity transform aligns the eye centers using uniform scale, rotation and translation.

The Earth-Moon composite joins oceans and clouds to lunar craters along an S-shaped boundary. Their similar outlines support this composite, although the same photographs do not produce a clear change of subject in the hybrid experiment.

The largest source contours above luminance 0.04 are fitted with cv2.fitEllipse and mapped affinely to a circle of radius 220 pixels centered at (256,256). The Earth mapping uses axis scale factors differing by approximately 1%; the Moon mapping is nearly uniform.

Both examples use the complete registered photographs, sampled with bilinear interpolation and replicated borders. Thresholding measures the planetary outlines only; the transformations sample the original RGB pixels. Exposure, color and backgrounds are unchanged, and no retouching or transfer of one background into the other source precedes blending.

Comparison methods

The hard cut, single-scale feathering and multiresolution blend use identical registered arrays and the same underlying binary selection. Feathering applies the exact coarsest Gaussian mask to the full images, so fine details and broad colors mix over the same wide transition. Multiresolution blending applies the corresponding Gaussian mask to each Laplacian band.

Initial mask smoothing

A binary finest mask leaves an abrupt texture change at the cat's crown and a contrast change where the planetary boundary meets the outer edge. Initial-mask Gaussian widths of 0, 2, 4, 8, 12 and 16 pixels are compared for each example. Sigma 8 softens these joins while keeping the tabby's eyes, nose and muzzle inside the selection; larger widths mix fine texture over a wider region.

Only the selection weights change; the source photographs and image detail bands remain fixed. The finest mask has sigma 8. Further smoothing at stack width sigma gives a nominal combined width of $\sqrt{8^2 + \sigma^2}$. The mask widths are therefore 8, 8.2, 8.9, 11.3, 17.9 and 33.0 pixels for the cat, with 64.5 pixels also used for Earth-Moon. The oraple retains a binary finest mask; supplementary figures show the crown and lower-edge comparisons.

Number of detail bands

The cat blend uses five detail bands with sigma = 2, 4, 8, 16 and 32 pixels. The Earth-Moon blend uses six, adding sigma = 64 pixels, and both include the coarse residual. Supplementary comparisons test four, five and six bands while keeping all other inputs fixed.

For the cat, a maximum sigma of 16 pixels leaves a more visible facial patch. Sigma 32 softens the coat boundary, while sigma 64 spreads the color transition farther into the surrounding fur. The Earth-Moon blend uses sigma 64 because the brightness mismatch extends across a large part of the disk.

Boundary appearance

The direct cut leaves a visible polygon boundary around the cat's face. Multiresolution blending reduces the coat-color change and retains more distinct tabby detail around the cheek and muzzle than broad feathering, which mixes both faces over a wider region. Clouds and craters also remain more localized at the Earth-Moon boundary. These observations concern the visible transitions in these two examples.

The boundary-jump measurement is the mean absolute RGB difference between adjacent pixels on opposite sides of the original binary contour. It includes natural image texture as well as the seam. Feathering gives smaller values because it mixes more detail, so a smaller jump alone does not establish a better-looking or physically realistic composite.

Color comparison

RGB is used for the final composites. The tabby's coat color separates its face from the white fur and orange ears, while blue oceans distinguish Earth from the nearly neutral Moon. These regions are less distinct in grayscale, although the boundary is still smoothed.

Limitations

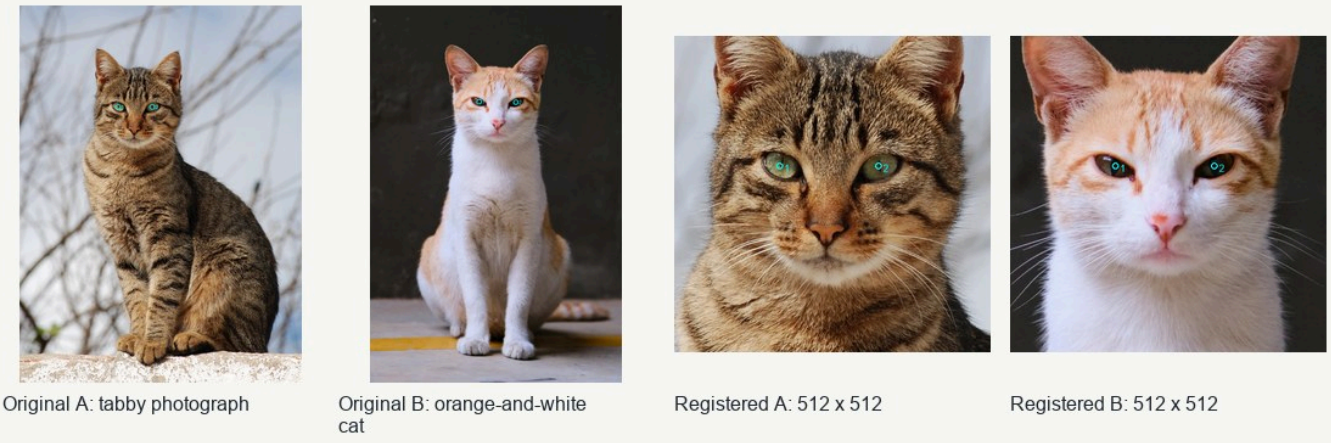
Multiresolution blending does not correct differences in anatomy or surface structure. The cats retain differences in muzzle shape and fur direction, and lunar features do not continue Earth's clouds. Smoothing the finest mask reduces the abrupt outer-edge join, but the Earth-Moon composite remains an imaginary surface.

Processing retains signed float64 values. Only the final display images are clipped to [0,1].

Blend settings and mean boundary differences

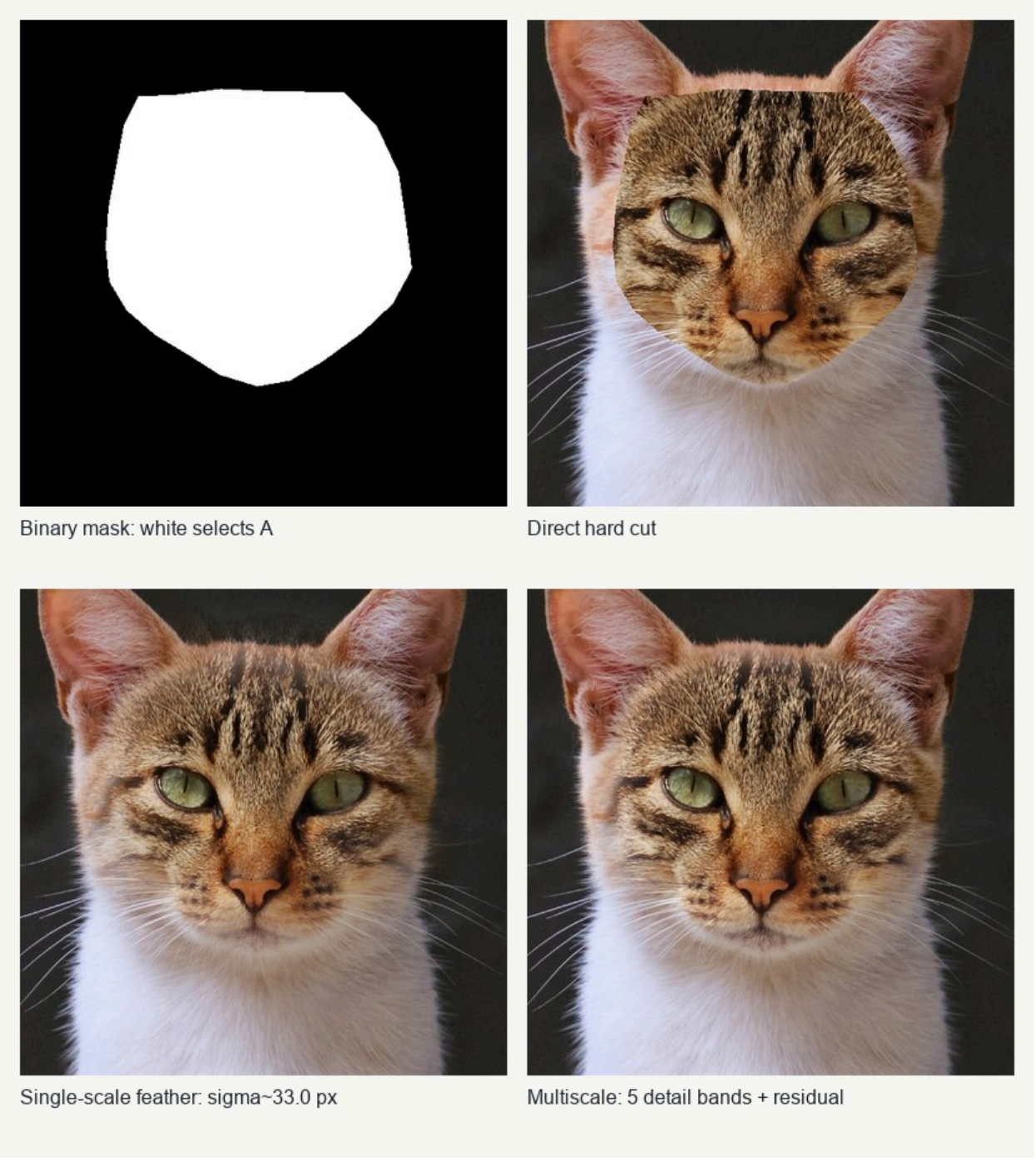
Parameter or measurement	Cat face blending	Earth-Moon blending
Selection mask	polygon	sinusoidal
Detail bands	5	6
Maximum image sigma (px)	32	64
Boundary jump: hard cut	0.2089	0.2718
Boundary jump: feathering	0.0289	0.0422
Boundary jump: multiresolution	0.0304	0.0424
Scalar values clipped	0.00%	4.92%

Cat face blending · Inputs and registration



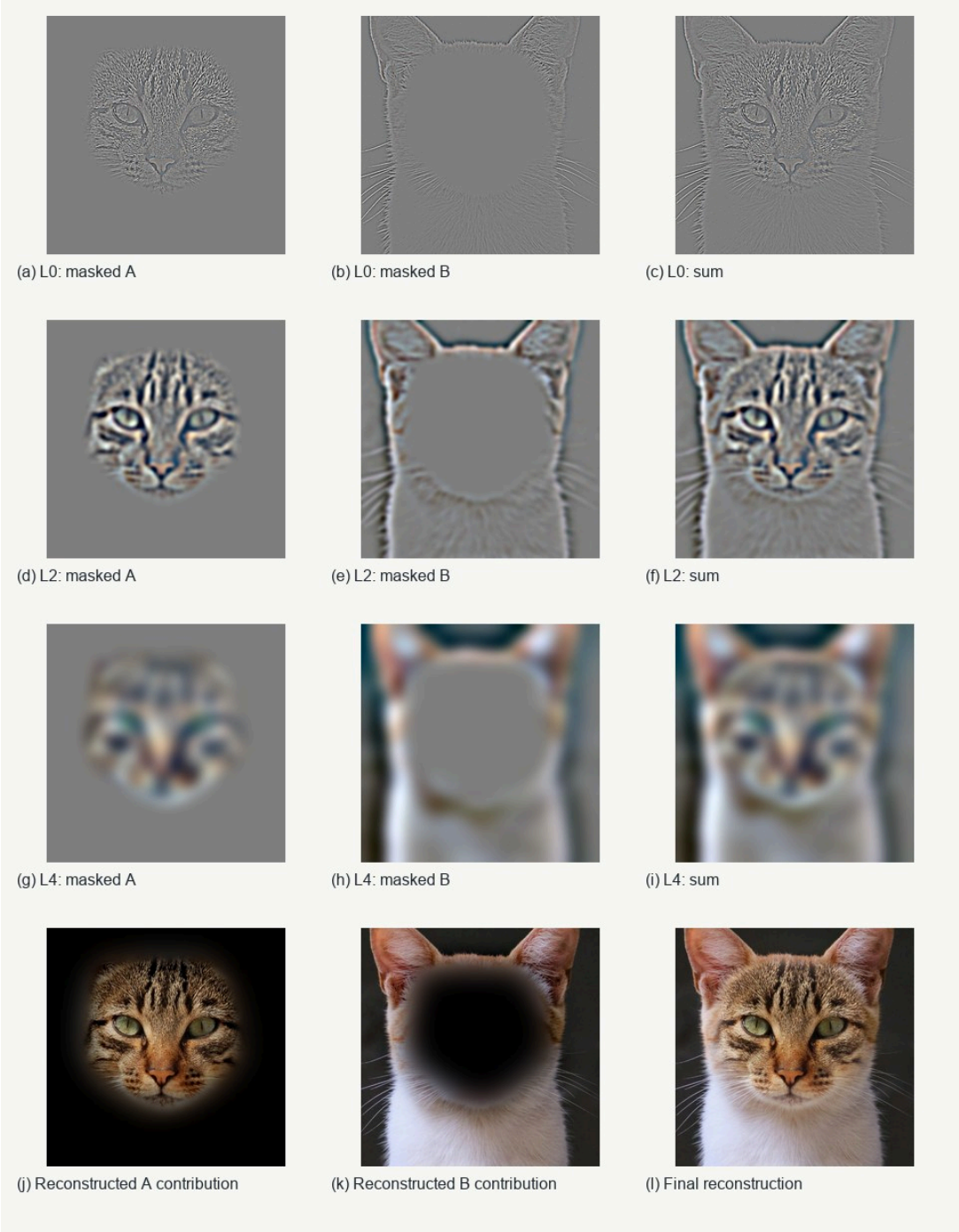
Cat face blending: original and registered photographs. Cyan circles mark the eye centers and their common output positions. Tabby photo: Joaquim Alves Gaspar (Alvesgaspar), CC BY-SA 3.0. Orange-and-white cat: Himanshu Choudhary, Unsplash License. The adapted composite and tabby-derived figures are distributed under CC BY-SA 3.0.

Cat face blending · Blending methods



Cat face blending: three methods using the same registered inputs and underlying binary selection. The initial mask is smoothed at sigma=8 pixels before the mask stack is constructed. The initial 8-pixel Gaussian reduces the abrupt texture change along the selection boundary. Tabby eye outlines, stripes, nose and muzzle remain distinct. Single-scale feathering combines the two coat textures over a wider region.

Cat face blending · Multiresolution contributions



Cat face blending: rows 1–3 show masked A, masked B and their sum at Laplacian levels 0, 2 and 4. The final row reconstructs each contribution and the result using all detail bands plus the coarse residual, following Burt–Adelson Figure 10. Each detail row uses one shared symmetric contrast scale; reconstructed rows retain actual intensity.

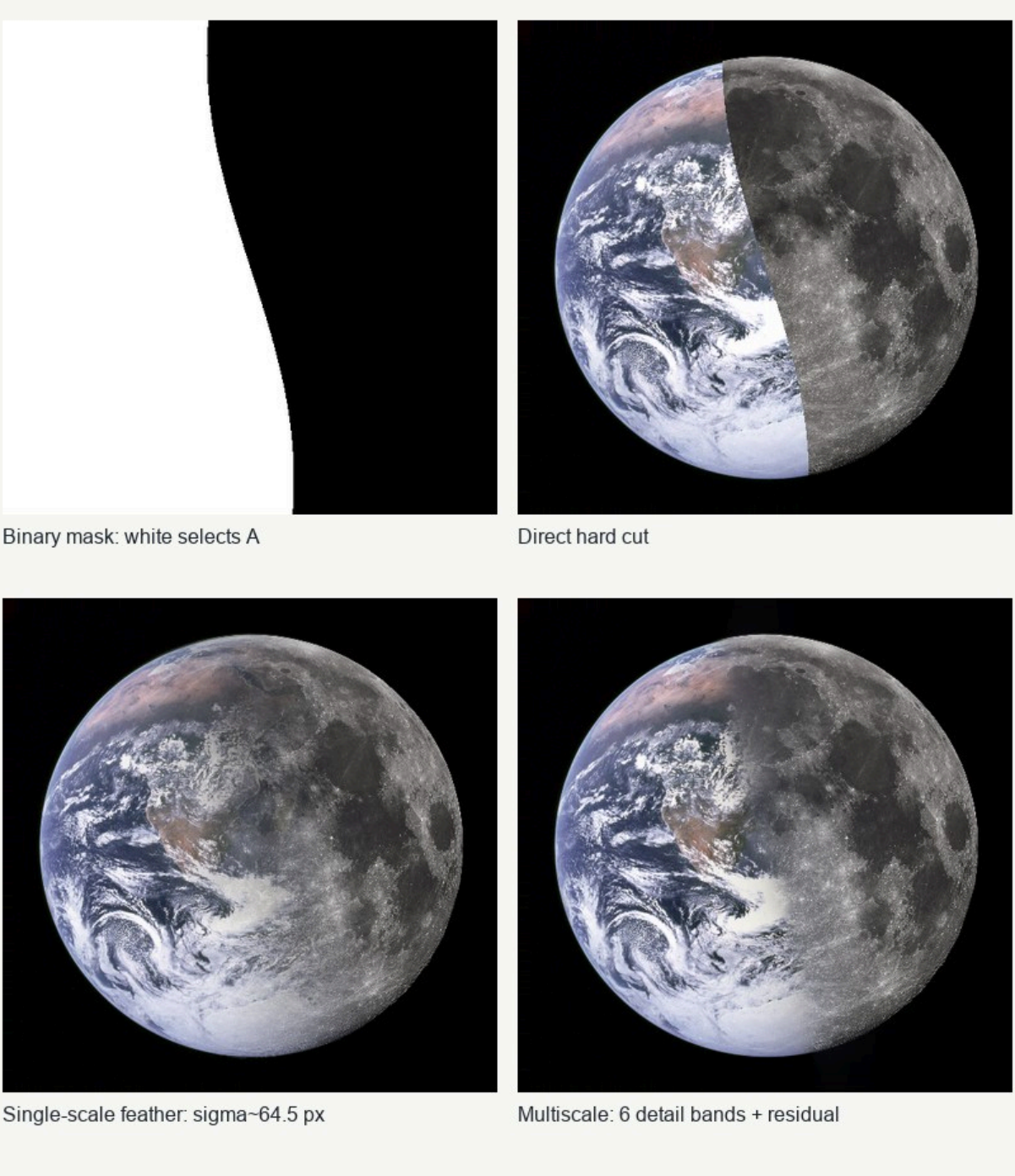
Earth-Moon blending · Inputs and registration



Original A: NASA Blue Marble | Original B: Gregory Revera's Moon | Registered A: 512 x 512 | Registered B: 512 x 512

Earth-Moon blending: original and registered photographs. Cyan outlines mark the fitted source ellipses and target circle. The larger axis scale factor exceeds the smaller by 1.06% for Earth and 0.03% for the Moon. Earth: NASA/Apollo 17, public domain. Moon: Gregory H. Revera, CC BY-SA 3.0. The adapted composite and Moon-derived figures are distributed under CC BY-SA 3.0.

Earth-Moon blending · Blending methods



Earth-Moon blending: three methods using the same registered inputs and underlying binary selection. The initial mask is smoothed at sigma=8 pixels before the mask stack is constructed. The brightness discontinuity at the direct cut is reduced. Smoothing the initial mask also reduces the abrupt change at the disk outline. Cloud and crater details remain more localized than with single-scale feathering.

Color experiment · Luminance and RGB



Cat face blending / luminance



Cat face blending / RGB



Earth-Moon blending / luminance



Earth-Moon blending / RGB

Luminance and RGB results with the same registration, masks and stack settings. Luminance is 0.2126R + 0.7152G + 0.0722B; RGB channels are blended independently.

Additional parameter comparisons are available in the online supplement.

Summary and observations

The main lesson from these experiments is that spatial scale determines which image structures are preserved. Smoothing reduces fine variation before differentiation. Hybrid images combine the low-frequency structure of one source with high-frequency detail from another. Multiresolution blending uses narrow transitions for fine detail and wider transitions for brightness and color differences.

The numerical tests establish consistency of filtering and reconstruction. Visual comparisons also show that these operations cannot fully resolve differences in facial geometry or source texture. Image registration and parameter selection therefore remain important even when reconstruction errors are close to floating-point precision.

Reproduction and image credits

The source package contains the input images, implementation, parameters, unit tests, recorded measurements, and report builder. The custom examples use the credited source photographs. The self-portrait and street photograph are from the supplied Assignment 1 images. The apple and orange inputs are the original PNGs from the supplied Part 2 image package. The Taj Mahal and Derek/Nutmeg photographs are also supplied course images. Source records, complete rights statements, and hashes are included in the image manifest.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
python run_filters.py
python run_frequencies.py
python build_report.py
python -m http.server 8000 --directory site
```

See README.md for image replacement, environment details, and PDF export. Numerical errors, recorded runtimes, and experiment settings are available in measurements.json.

Image	Credit	Rights / provenance	Link
Derek portrait	Derek Hoiem / course materials	Course sample; see note below.	Source
Nutmeg the cat	Derek Hoiem / course materials	Course sample; see note below.	Source
Taj Mahal	Course materials; individual photographer not specified	Course sample; see note below.	Source
Classic cameraman	Course materials; individual photographer not specified	Course sample; see note below.	Source
Apple (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Orange (supplied Burt-Adelson original)	Peter J. Burt and Edward H. Adelson / course materials	Course sample; see note below.	Supplied Part 2 image package
Moksh portrait	User-supplied photograph	User-provided photograph.	Supplied photograph
Street photograph	User-supplied photograph	User-provided photograph.	Supplied photograph
Astronaut Eileen Collins portrait	NASA	Public domain (NASA photograph)	Source License
The Blue Marble / Earth seen from Apollo 17	NASA / Apollo 17 crew	Public domain (U.S. government work)	Source License
Full Moon, 22 October 2010	Gregory H. Revera	CC BY-SA 3.0	Source License
Barack Obama official portrait, December 2012	Pete Souza / White House	Public domain (U.S. government work)	Source
Young male tabby cat, Portugal, November 2010	Joaquim Alves Gaspar (Alvesgaspar)	CC BY-SA 3.0	Source License
Front-facing orange-and-white cat, Bengaluru	Himanshu Choudhary (@aalochak)	Unsplash License	Source License

Course samples are supplied for this assignment; no general open license is asserted. Personal photographs are user-provided. Adapted figures containing the Moon photograph by Gregory H. Revera or the tabby photograph by Alvesgaspar remain under CC BY-SA 3.0, with full attribution recorded in the image manifest.

References

1. A. Oliva, A. Torralba, and P. G. Schyns. Hybrid images. ACM Transactions on Graphics 25(3), 527–532, 2006.
2. P. J. Burt and E. H. Adelson. A multiresolution spline with application to image mosaics. ACM Transactions on Graphics 2(4), 217–236, 1983.
3. R. Szeliski. Computer Vision: Algorithms and Applications, second edition. Figure 3.42.
4. SYDE 671 Assignment 2, “Fun with Filters and Frequencies.” Required experiments and extensions follow the supplied handout, adapted from Berkeley CS180.